ENAE450 Final Report

Paul Gomez Wick (pgomezwi@terpmail.umd.edu)
Jonathan Jayaputra (jjayaput@terpmail.umd.edu)
Shreena Thakker (shreenat@terpmail.umd.edu)

May 15, 2026

Introduction

The goal of this project was to navigate a Turtlebot 4 robot out of a maze. The first phase of the project involved navigating a complex maze, with three minutes prior to our runs to create a map of the environment. The robot would then be placed randomly within the maze, use the saved map to navigate, and turn 180° once outside. The second part of the project involved navigating a simpler maze (without loops), but without a pre-mapping stage.

This lab report gives an overview of our process and approach towards the problem, the tools and techniques that we used, an analysis of our results, and a reflection of our experience.

Hardware

The TurtleBot 4 is an open source robotics platform primarily marketed and used for educational and research purposes. It runs on a Raspberry Pi 4B, with 4 GB of RAM, though we did not focus on using its on-board processing and storage capabilities beyond communicating with it via ROS2 topics, and it runs Ubuntu 20.04. The main sensor we used was its 2D LIDAR system (seen in Figure 1 as a small disc on top of the body), which is rated for use in distances of 0.15 to 12 meters, with an 8kHz sampling rate, 360° field of view, and an angular resolution of one degree. It uses two drive motors for locomotion, with a top speed of 0.31 m/s, and a top angular speed of 1.9 rad/s. Due to the fact that we only used the LIDAR system for sensing, we were limited in both how fine of a resolution we could achieve, due to the LIDAR's resolution, as well as not being able to achieve fine-resolution distance measurements below 15cm, which forced us to use more generous obstacle avoidance parameters to not run into the walls.

Technical Discussion

As a preface, we did not get our nodes running on the physical robots, so we will primarily be discussing results simulated in Gazebo.

Figure 1: The TurtleBot 4

Pre-mapped Environment

For the pre-mapped environment navigation part of the project, we used SLAM (Simultaneous Localization and Mapping) and Nav2 to generate a map and autonomously navigate the maze. This part of the project was broken down into two sections: the mapping phase and the navigation phase.

During the mapping phase, we used teleoperation and SLAM to create a 2D occupancy grid map of the environment using LiDAR scans and robot odometry, saving this map with Nav2's `map_saver_cli`.

For the navigation phase, we used the Nav2 autonomous navigation stack to escape the maze which directs the robot to the goal position and orientation with built-in obstacle avoidance. For initial localization, we used AMCL (Adaptive Monte Carlo Localization) to estimate the position and orientation of the robot relative to the saved map. As the robot moves, AMCL refines this estimation by comparing the incoming LiDAR with the saved map and uses a probabilistic particle filter method to estimate the robot's most likely pose.

Once the initial position is received and AMCL publishes its estimate transform between the map and the robot, Nav2 is able to create a costmap, a grid representation of the environment with a "cost" value assigned to each cell. This costmap is then used to determine a path from the robot's current position to the goal position while minimizing the cost. Nav2 continuously generates velocity commands to the `cmd_vel` topic and adjusts movement to avoid obstacles and stay on path towards the goal, using LiDAR sensors to detect any obstacles missed in the map.

To set the goal position, we used RViz's Nav2 Goal tool to send a goal pose at the exit of the map, turned 180 degrees.

Pre-mapped Runs

We used a custom generated map for our runs. After the mapping phase, we saved the map image in Figure 2. Using this map, we did two runs, shown in Figures 3 and 4.

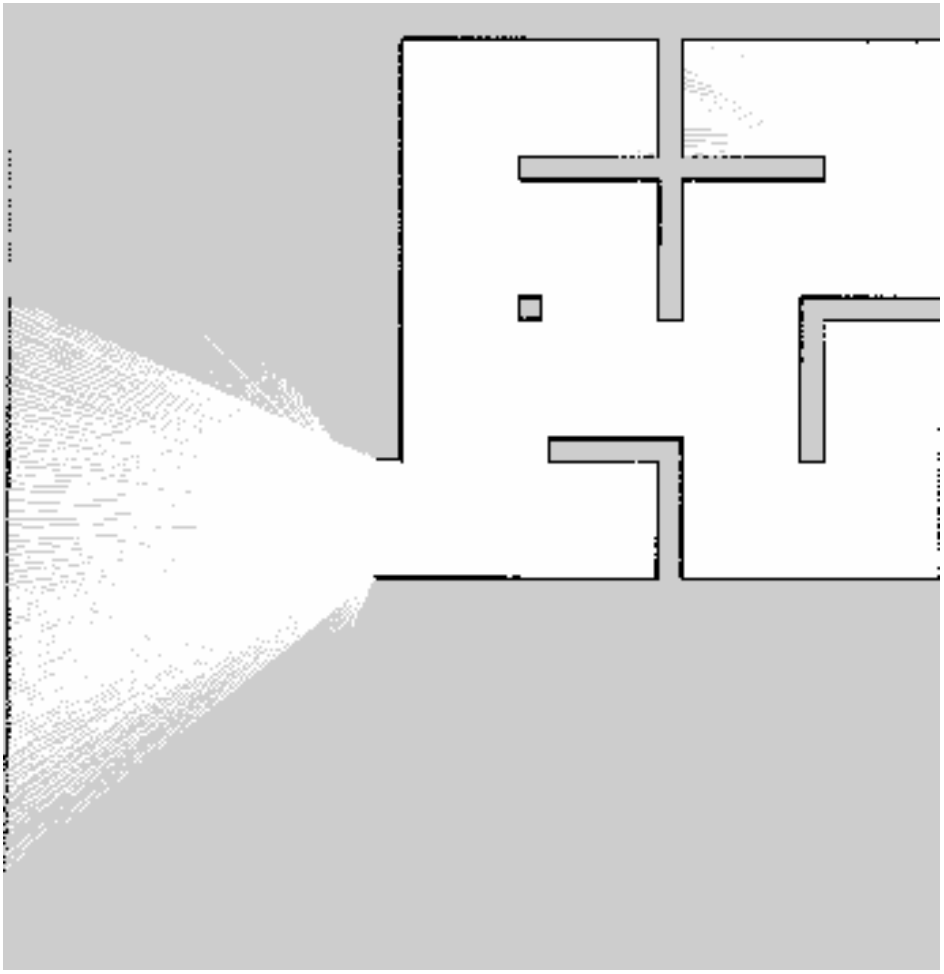

Figure 2: Pre-mapped maze

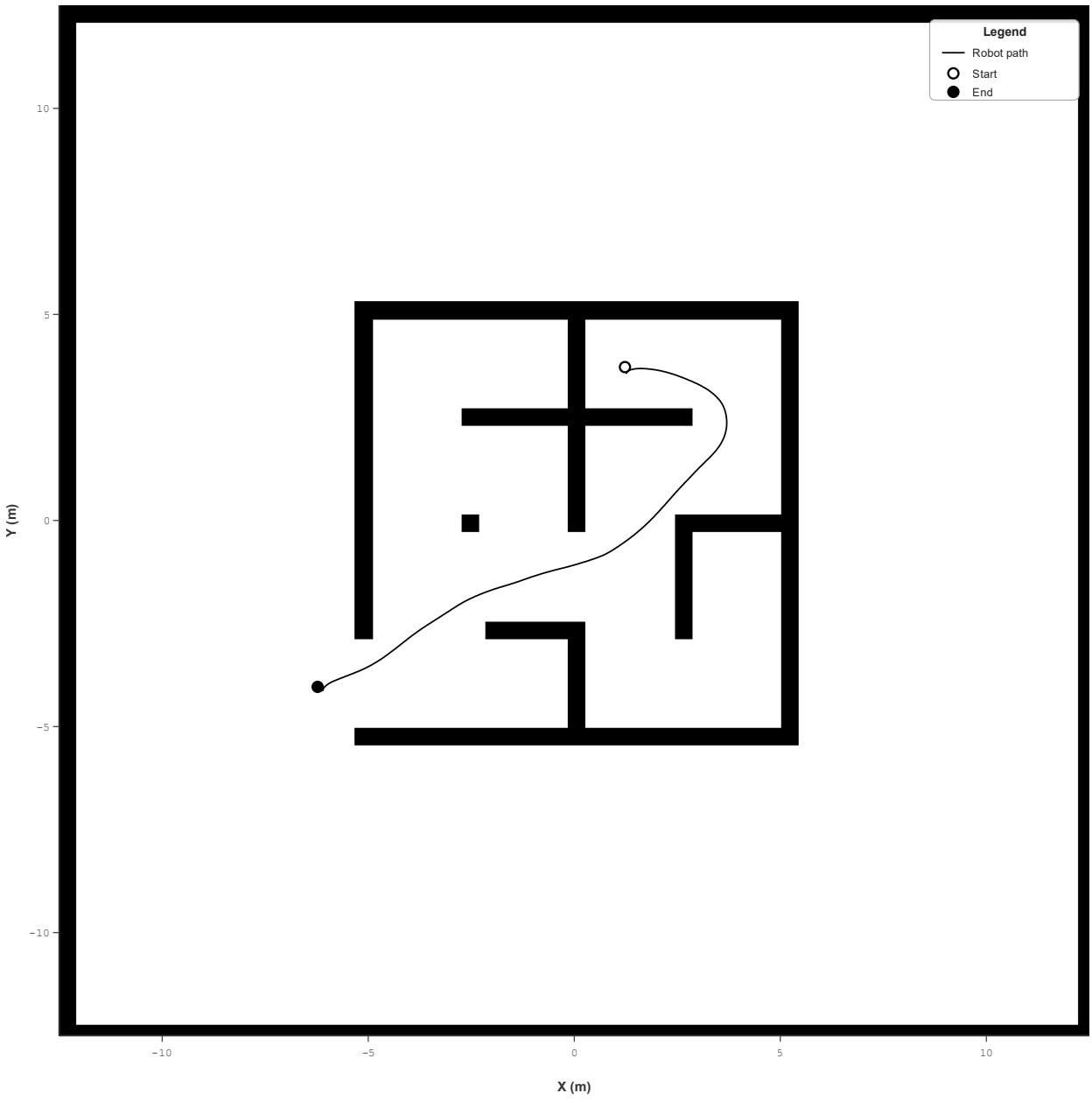

Figure 3: First premapped-run

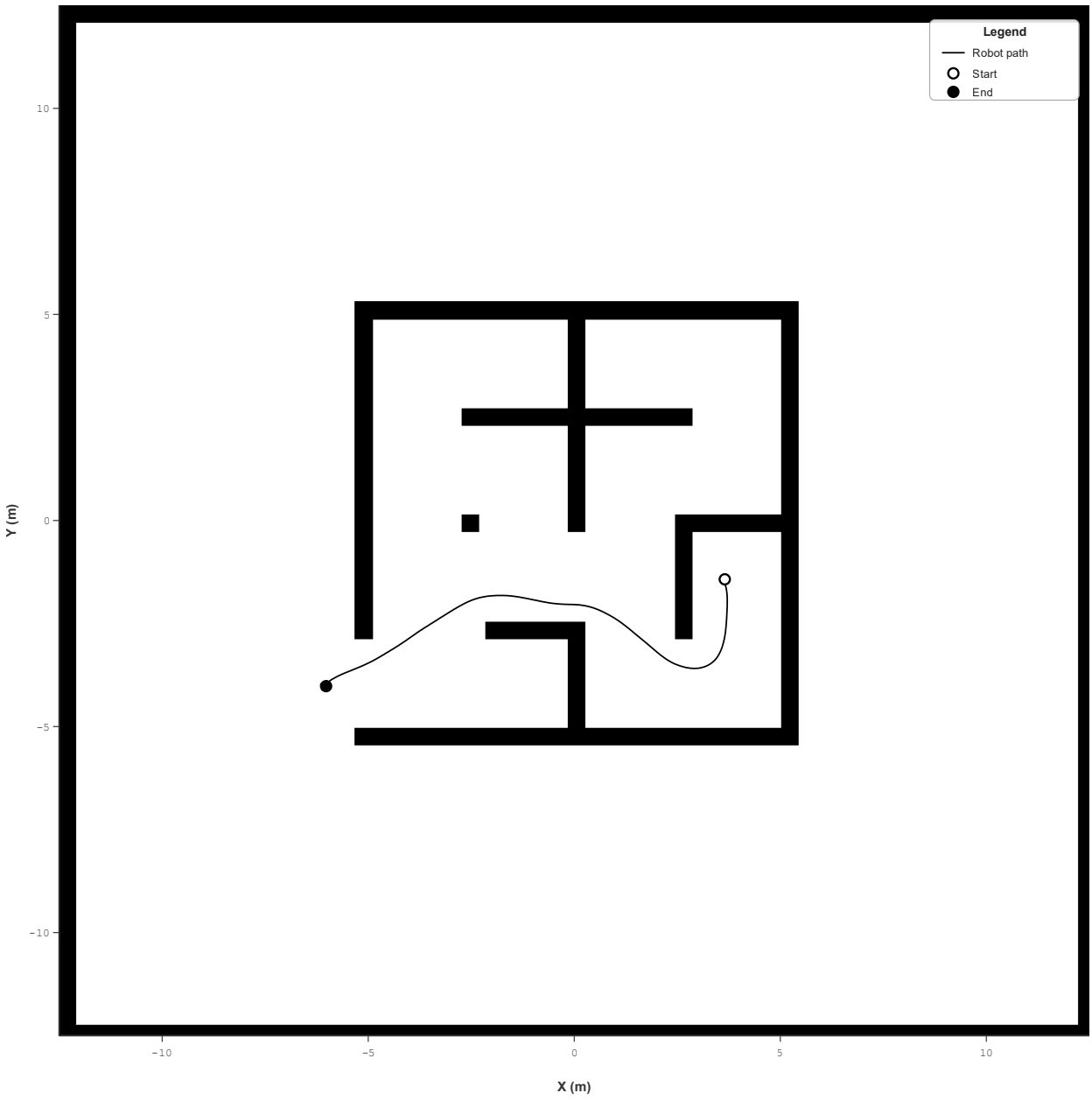

Figure 4: Second premapped-run

Launch Files

For the mapping phase, we used the `tb4_simslam_launch.py` file provided to launch the simulation with SLAM running (Conroy, 2026).

After we saved the SLAM map, we shut down the previous simulation and used a modified version of `tb4_sim_launch.py` to get AMCL and Nav2 working with the TurtleBot simulator, which can be found in our `kill_order/simulation/launch` folder(Conroy, 2026). This launch file runs the simulation with the actual map file and automatically launches RViz with the saved SLAM map. Once the user inputs an initial pose through RViz, the launcher will spin up the Nav2 stack, which uses the SLAM map to create global and local costmaps.

Novel Environment

Our approach for novel environment exploration was to use a right-hand wall follower, which is effective in that it will always find the exit to the maze but loses time taking detours. An example of this can be seen along the negative diagonal of Figure 5, with a long section of navigation which is backtracked over.

The Navigation Process

We used a finite state machine to dictate what the TurtleBot's actions. We provide a UML state diagram in Figure 6 and will give a high-level overview here.

[**START**] The robot, when initially placed, will try to orient itself towards the most open space using the LIDAR scan. It begins by first finding the angle, `pinch_angle`, of the shortest distance, `happy_distance`, between two rays opposite from each other and takes this to be the width of the hallway, perpendicular to the walls. Then, for the two angles perpendicular to `pinch_angle`, it chooses the angle which casts a further ray as its `turning_goal` (also adding the current yaw, as measured by the odometry).

[**TURN_IN_PLACE**] Our turning code uses PD control to approach the `turning_goal`, using the difference between the yaw of the current pose and `turning_goal`, continuing when the proportional error is sufficiently low. Right now, the robot will turn towards the open hallway.

[**NAVIGATE_TO_WALL**] Intuitively, in order to follow the right wall, we need to know where the right wall is. To map out the TurtleBot's surroundings, we use an adapted laser scan processor to extract the line segments out of the scan (Riccardi, 2020). But this isn't useful immediately, as the scan processor node may not have published a good reading yet. So, while the TurtleBot doesn't have a line segment directly to its right to follow, it uses a simple PD control loop to try to stay in the middle of the hallway and angled parallel to the wall using the current `pinch_angle` (which conveniently helps take simple turns, if needed).

[**NAVIGATE_WITH_WALL**] Eventually, there will be a sufficient amount of consecutive successful right wall reads where the TurtleBot can transition to direct wall following. Here, it uses PD control to stay a certain distance away from the right wall (in our case it is about half of `happy_distance`).

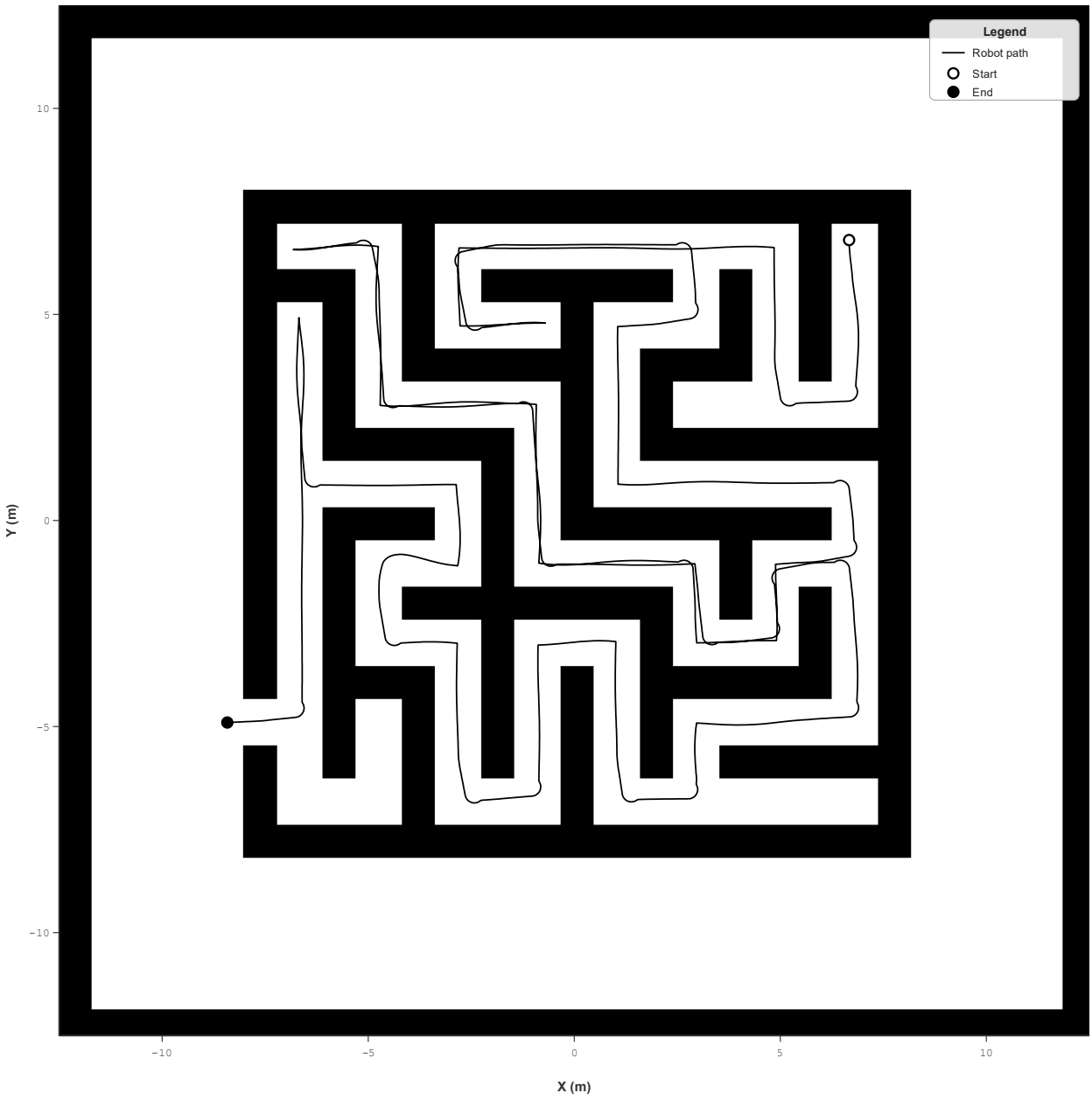

Figure 5: TurtleBot's navigation path on a custom map

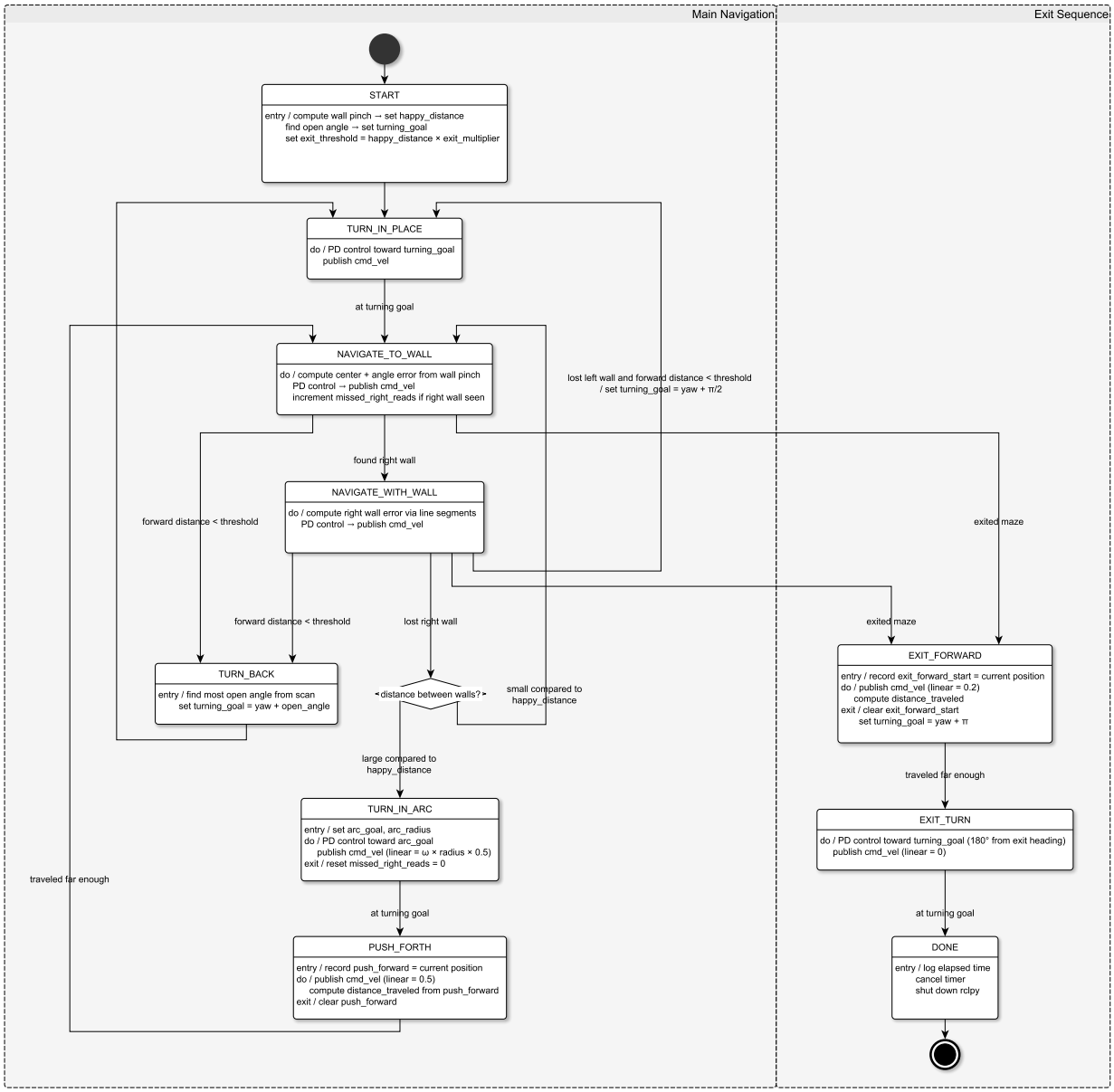

Figure 6: Wall follower state diagram

The TurtleBot, however, can not follow a right wall forever. Eventually it will reach a corner of some sort and has to make an executive decision on what to do. There are five cases we are interested in here.

The first two both start when the TurtleBot loses track of the right wall, which could be for two reasons: the processor node is being fickle or the right wall is actually gone. We determine this by finding the current `pinch_distance` and comparing it to the first measured `happy_distance`. If `pinch_distance` is not significantly different from `happy_distance`, we decide that this is probably the fault of the processor node, and return to the [NAVIGATE_TO_WALL] state. Otherwise, there is probably a right turn which we have to make, and we transition to [TURN_IN_ARC].

The third is when there is no left wall found and the forward distance is sufficiently low, indicating a left turn ahead. In this case, we simply set the `turning_goal` to the current yaw plus $\pi/2$ `rad` and return to [TURN_IN_PLACE].

The fourth is when the forward distance is significantly low, indicating a dead end, as otherwise the previous state transitions should have already occurred. In this case, we transition to [TURN_BACK].

The fifth is when a significant portion of the scan in front of the TurtleBot is much greater than the first measured `happy_distance`. In this case, we are pretty sure we have found the exit and transition to [EXIT_FORWARD].

Note: The checks for [TURN_BACK] and [EXIT_FORWARD] also occur in [NAVIGATE_TO_WALL], though they are unlikely to be used.

[**TURN_IN_ARC**] Turning in an arc is similar to turning in place, but with a linear velocity. It also uses PD control to turn towards the desired angle, but adds a linear x-component to the published Twist. After having completed the turn, we want to go forward a little bit to ensure we are in a hallway.

[**PUSH_FORTH**] This state mostly exists as a safety measure to prevent weird readings from the navigation control loops. The robot will simply move forward a certain distance, then transition to [NAVIGATE_TO_WALL].

[**TURN_BACK**] Here, we want to find the largest open angle. We simply find the angle (window of angles) with the furthest reading and set that as our `turning_goal` before transitioning to [TURN_IN_PLACE].

[**EXIT_FORWARD**] Once the TurtleBot has found the exit to the maze, it will drive forward until it reaches a certain distance.

[**EXIT_TURN**] It will then rotate back to face the maze and stop.

[**DONE**] At last, the TurtleBot will stop its timer callback and shut down `rclpy`.

Assumptions

While we have control over the map, we tried to minimize the number of hard-coded values, so most distance values are based on `happy_distance`. Theoretically, this allows the navigator node to be fairly robust in different environments as long as the hallway widths remain somewhat consistent throughout the maze, though we did not test this thoroughly. It was also beneficial to increase the wall thickness, as U-bends could be troublesome sometimes otherwise.

Reflection

We would have liked to get our code running on the actual robots, but due to computer issues and time constraints, we could not do so. Additionally, the line segment extractor seemed to have lower performance in smaller simulated mazes and may not have been as successful in lab as it was in simulation.

A simple feature we would have liked to implement, given more time, would be to have the node alternate right hand and left hand wall following between runs to reduce variance caused by the starting position. In the starting position shown in Figure 5, this could have skipped two of the long backtracking sections.

More complex would have been to have the first run use our wall-follower algorithm to exit the maze while also running SLAM to map the environment. Because wall following is equivalent to depth-first search for mazes without loops, the first run is likely to get a pretty good read of the environment. Then, our second run would be able to use Nav2 goals to escape the maze much more efficiently. We would also have a conditional launcher that checks if there is a saved map and spins the according navigation stack.

In total, we are very pleased with what we were able to accomplish, especially with getting both of the phases working in the simulation.

Team Contributions

Paul Gomez Wick: Adapting novel environment code to simulation

Jonathan Jayaputra: Pre-mapping launch files and package adaptation, novel environment package

Shreena Thakker: Pre-mapping launch files and package

References

Clearpath Robotics. (2026). *Turtlebot4.* https://clearpathrobotics.com/turtlebot-4/. (Accessed May 14, 2026)

Conroy, J. (2026). *Turtlebot simulator.* https://code.umd.edu/conroyj/turtlebot4_2d_sim. (Accessed May 15, 2026)

Open Navigation LLC. (2026). *Nav2 docs.* https://docs.nav2.org. (Accessed May 15, 2026)

Riccardi, A. (2020). *Laser scan processor.* https://github.com/AleRiccardi/laser_scan_processor. (Accessed May 12, 2026)

(Riccardi, 2020) (Open Navigation LLC, 2026) (Conroy, 2026) (Clearpath Robotics, 2026)